



Universidad de Jaén

Bloque 2

El Proceso del Software



EPS
Escuela Politécnica
Superior de Jaén

Objetivos
Introducción
Modelos de proceso
El Proceso Unificado
Estrategias ágiles
¿Qué método es el más adecuado?

Tema 2. Modelos del Proceso

L. Alfonso Ureña López Eugenio Martínez Cámara

Departamento de Informática
Universidad de Jaén

Fundamentos de Ingeniería del Software
Grado en Ingeniería Informática
2º Curso

Contenidos

- 1 Objetivos
- 2 Introducción
- 3 Modelos de proceso
 - El modelo en cascada
 - El modelo incremental
 - El modelo DRA
 - Prototipado
 - El modelo en espiral
 - Desarrollo basado en componentes
 - El modelo de métodos formales
- 4 El Proceso Unificado
 - Introducción
 - Características
 - Fases del proceso
- 5 Estrategias ágiles
- 6 ¿Qué método es el más adecuado?

Objetivos

- Introducir los **modelos del proceso** de desarrollo del software
- Describir el uso y **características** de cada uno de los modelos
- Hacer una **descripción** de los modelos indicando su **aportación** al análisis de requisitos, el desarrollo del software, la fase de pruebas y el mantenimiento
- Definir los roles del **prototipado** en el proceso de desarrollo de software
- Conocer la **historia** del **proceso unificado** y de la aparición de **UML**
- Conocer las **fases** del **proceso unificado** y la situación del análisis de requisitos en dicho proceso

Introducción

- Un **proceso** de desarrollo de **software** establece un **conjunto de actividades** que conducen a la **creación** de un **producto software**
- La **creación de software** es **difícil** de **automatizar**. Aunque existen herramientas automáticas para la Ingeniería del Software (herramientas **CASE**) es difícil conseguir actualmente mayor automatización. Esto es debido a que la creación de software es una labor que **depende** del **juicio humano**, y por tanto, presenta una inmensa diversidad de posibilidades

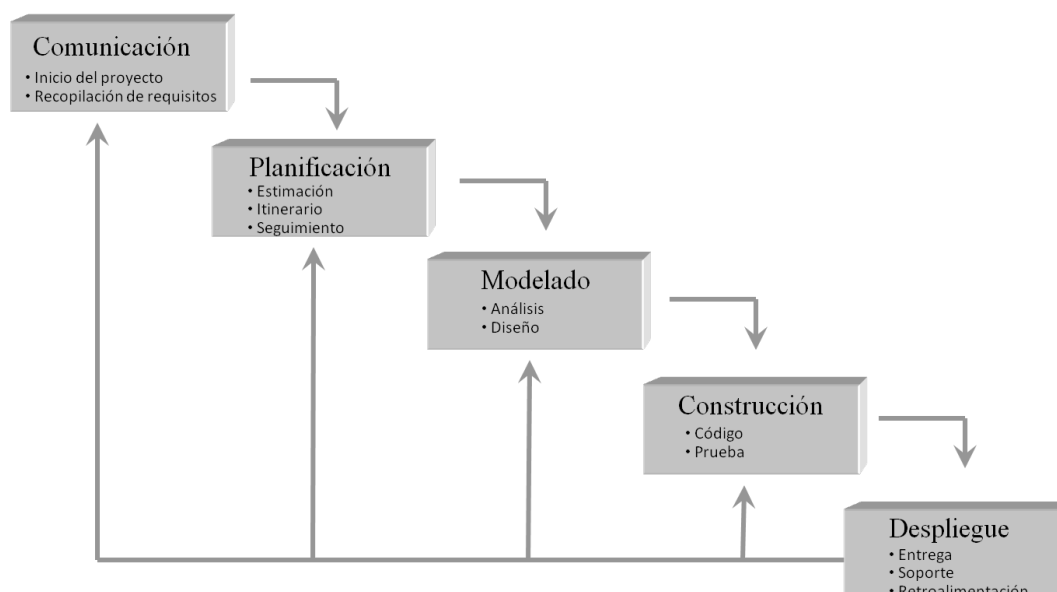
Introducción

- Por estas razones, no existe un proceso de desarrollo de software ideal. Este **proceso depende** en gran medida del **tipo de proyecto**, de su tamaño y de la organización que lo desarrolla
- En esta lección se presentan un conjunto de **modelos de procesos** prescriptivos, así como sus principales **características**

El modelo en cascada

- Es el paradigma **más antiguo**. Suele denominarse de estas otras formas: ciclo de vida clásico, modelo de las fases y modelo lineal secuencial
- Divide el proceso de software en **etapas**:
 - **Comunicación**: inicio del proyecto, recopilación de requisitos
 - **Planificación**: estimación, itinerario, seguimiento
 - **Modelado**: análisis y diseño
 - **Construcción**: código y prueba
 - **Despliegue**: entrega, soporte y retroalimentación

El modelo en cascada



El modelo en cascada



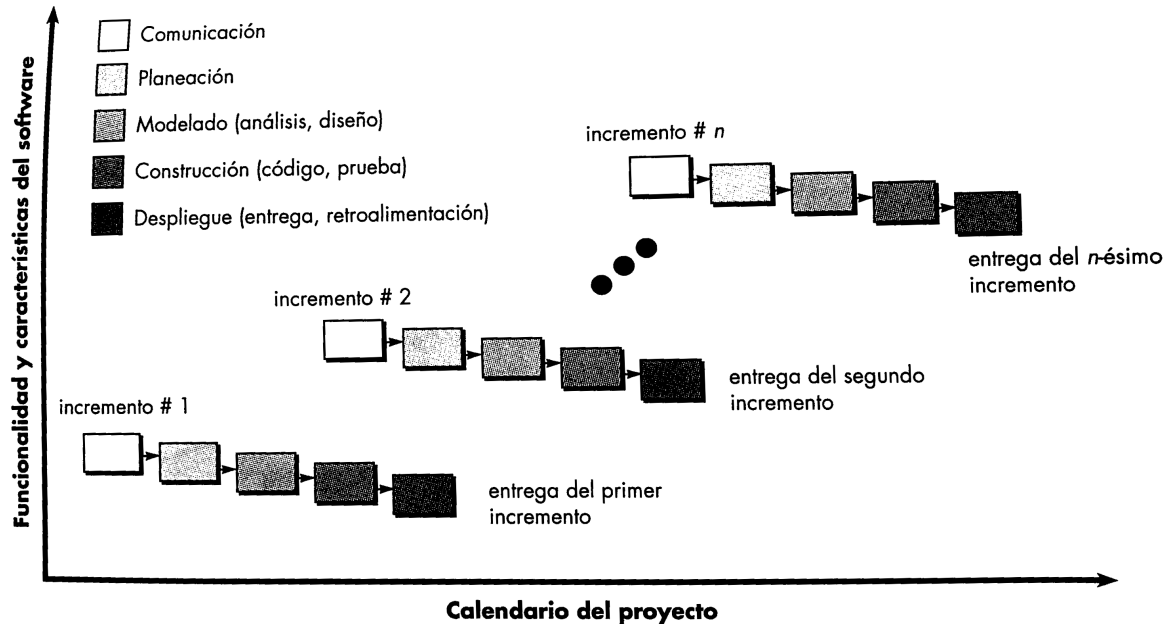
Inconvenientes

- Los proyectos reales rara vez siguen un modelo secuencial. El proceso de definición y **desarrollo** de **software** es altamente **no-lineal**
- Es difícil que el cliente exponga todos los requisitos al inicio. Este modelo **no acomoda** de manera apropiada la **incertidumbre** propia del **comienzo**
- **Pocos proyectos** tienen un **conjunto estable** de **requisitos**
- El **cliente** debe tener paciencia. El cliente no dispondrá de **ninguna versión** hasta **etapas muy avanzadas** del proyecto. Esta espera puede dar lugar a **incertidumbre** sobre el éxito del mismo

El modelo incremental

- En lugar de entregar el producto en su totalidad, éste se divide en **incrementos** de tal modo que **cada entrega** es un producto operativo pero incompleto
- Los **incrementos** se estructuran de modo que los primeros incluyan los **requisitos más importantes** del producto
- Cada **incremento** es desarrollado en un **periodo** de tiempo determinado y **breve**
- Normalmente, una vez ha comenzado el **desarrollo** de un **incremento**, sus **requisitos no se modifican** hasta que ha terminado. Los **cambios** a este incremento o a otra parte del producto se pueden incluir en la planificación de un **incremento posterior**

El modelo incremental



El modelo incremental



Ventajas

- Los **clientes** pueden hacer **uso** de un **software** con las características primordiales en **etapas tempranas**
- Se pueden utilizar los **primeros incrementos** como prototipos para obtener **información** de los mismos
- El **riesgo** de que el proyecto falle se **disminuye** con respecto al modelo en **cascada**
- Los **incrementos prioritarios** del sistema son los **más probados**



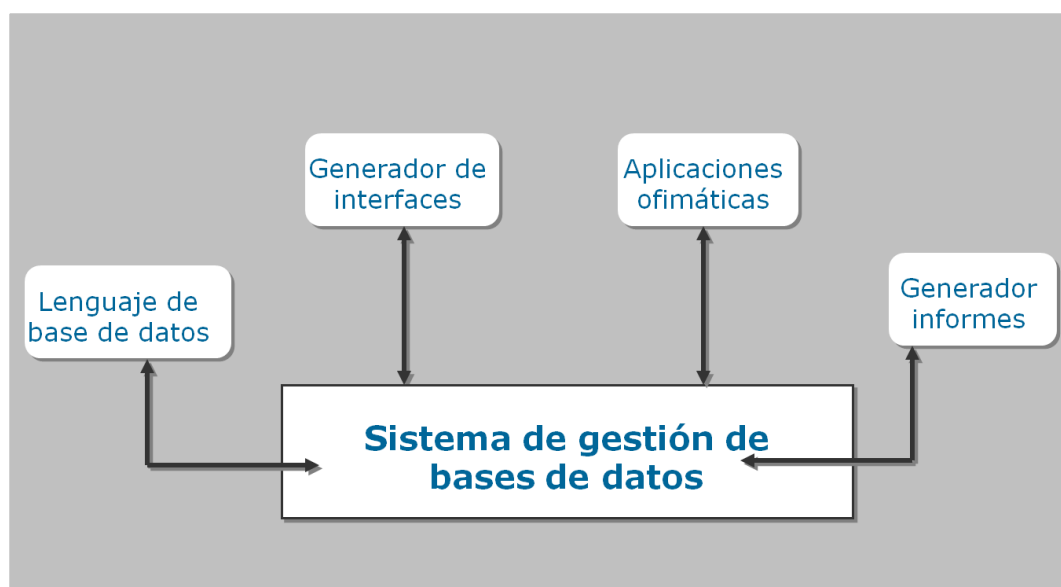
Inconvenientes

- **Dificultad** a la hora de acomodar **determinados requisitos** en un incremento
- **Dificultad** para determinar la **parte común**

El modelo DRA

- Los **métodos ágiles** están teniendo muchísima importancia e **influencia** en los **últimos años** pero otros métodos como el **Desarrollo Rápido de Aplicaciones (DRA)** se vienen utilizando desde hace muchos años
- Es una aplicación a **alta velocidad** del **modelo en cascada**
- Se basa en la utilización de **componentes ya existentes** y en la creación de **componentes reutilizables**
- Este enfoque está especialmente diseñado para el desarrollo de **aplicaciones** para negocios que hacen un **uso intensivo de datos**
- Los sistemas basados en **DRA** han tenido un **gran éxito** debido a que las **aplicaciones de negocios** basadas en la manipulación y presentación de información procedente de una **base de datos** tienen **numerosas partes en común**

El modelo DRA



El modelo DRA



Ventajas

- Este enfoque permite un **desarrollo muy rápido** de **aplicaciones** relativamente **sencillas**
- **Gran parte** del código ya está **implementado**



Inconvenientes

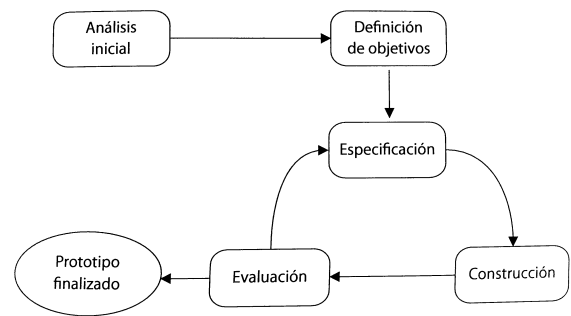
- En **sistemas grandes**, puede ser **complejo organizar** a todos los **equipos**
- **Dificultad** al implementar **interfaces no estándares**
- **Problemas de rendimiento** al **cargar** en memoria funcionalidad **no necesaria**

Prototipado

- En el **modelo en cascada** el usuario **sólo** comprueba realmente cómo **funciona** en la **entrega final**
- A veces los **requisitos** no están claros: el **prototipado evita** muchas de las **equivocaciones y ambigüedades** que puedan existir en los requisitos
- Una de sus utilidades puede ser el **estudio** de la **interfaz hombre-máquina**, para decidir qué debe mostrarse y qué debe introducir el usuario
- Un **prototipo** se **diferencia** del **sistema final** en que se encuentra **inacabado** y tiene una **construcción menos elástica**
- Suelen tener **limitada** la capacidad de **procesamiento de datos**, tener un pobre **rendimiento** y una limitada **calidad**
- **Herramientas** de prototipado rápido: <http://www.prototypingtools.co>

Principales etapas del prototipado

- Realizar un **análisis inicial**
- Definir los **objetivos** del prototipo
- Repetir
 - **Especificar** el prototipo
 - **Construir** el prototipo
 - **Evaluar** el prototipo y **recomendar cambios**
- Hasta que el **prototipo** esté **finalizado**



Prototipado



Ventajas

- Las **demostraciones tempranas** ayudan a **identificar malentendidos** entre el diseñador y el cliente
- **Identificar requisitos** que se hayan podido pasar por alto
- **Identificar dificultades** en la **interfaz**
- Comprobar **viabilidad** y **utilidad** del **sistema** aunque el prototipo no sea un modelo completo por naturaleza

Prototipado



Inconvenientes

- El **cliente** puede percibir el **prototipo** como **parte** del **sistema**
- Puede **apartar** la **atención** de los **asuntos funcionales** para **centrar** la atención en temas de la **interfaz**
- Requiere una importante **implicación** del **usuario**
- Gestionar su ciclo de vida requiere una **toma** de **decisiones** extremadamente **cuidadosa**

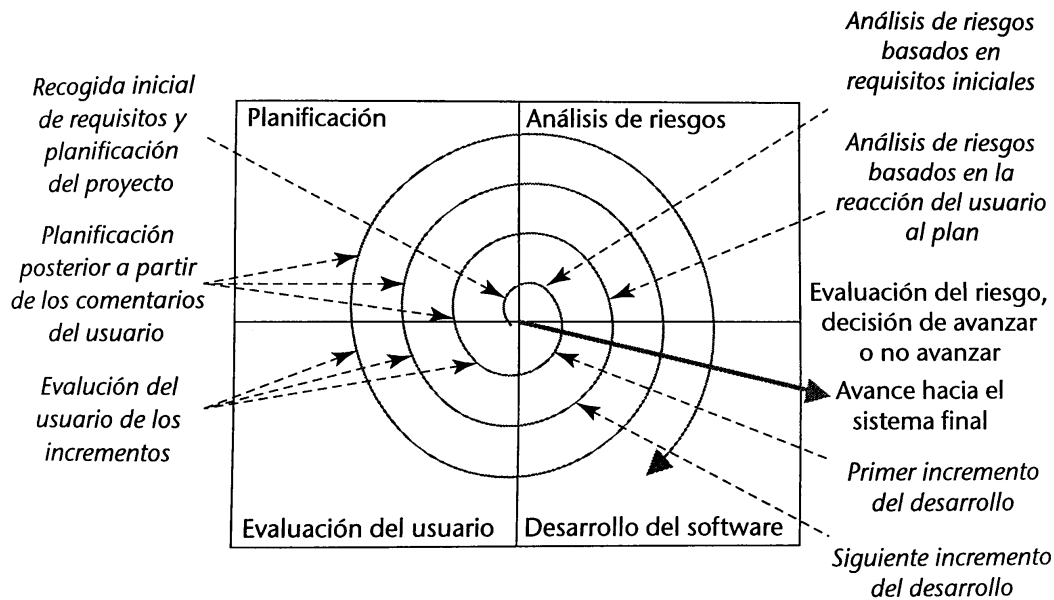
El modelo en espiral

- El **proceso de desarrollo** se representa como un **espiral** en lugar de como una secuencia de pasos
- **No** existen **fases predeterminadas** en este modelo tales como **análisis** o **diseño**
- El primer circuito alrededor de la **espiral** quizás genere la **especificación** del producto, los pasos siguientes se pueden aprovechar para desarrollar un **prototipo** y después se entregan **versiones más elaboradas** de manera progresiva
- Otra característica que lo distingue del resto de modelos es la presencia en el mismo de un **análisis de riesgos** de manera **explícita**

Objetivos
Introducción
Modelos de proceso
El Proceso Unificado
Estrategias ágiles
¿Qué método es el más adecuado?

El modelo en cascada
El modelo incremental
El modelo DRA
Prototipado
El modelo en espiral
Desarrollo basado en componentes
El modelo de métodos formales

Modelo en espiral para la entrega incremental



L. Alfonso Ureña López, Eugenio Martínez Cámara

2. Modelos del Proceso

Objetivos
Introducción
Modelos de proceso
El Proceso Unificado
Estrategias ágiles
¿Qué método es el más adecuado?

El modelo en cascada
El modelo incremental
El modelo DRA
Prototipado
El modelo en espiral
Desarrollo basado en componentes
El modelo de métodos formales

Desarrollo basado en componentes

- Su principal característica es la **reusabilidad**
- **Etapas** del proceso:
 - **Análisis** de componentes
 - **Modificación o adaptación** de requisitos
 - **Diseño** del sistema con **reusabilidad**
 - **Desarrollo e integración**
- Este enfoque está siendo cada vez más utilizado debido al **auge** de **componentes estándares**

L. Alfonso Ureña López, Eugenio Martínez Cámara

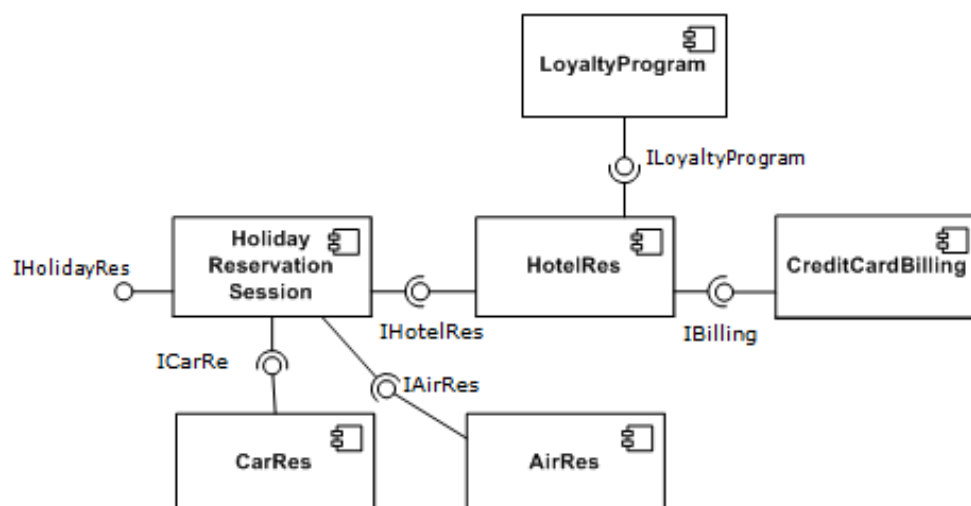
2. Modelos del Proceso

Desarrollo basado en componentes

- Necesidades:
 - Una biblioteca de componentes
 - Los componentes deben tener una estructura consistente que permita su interacción y adaptación
- Existen algunos estándares:
 - OMG/CORBA
 - Microsoft COM
 - Sun JavaBeans

Desarrollo basado en componentes

Ejemplo



Desarrollo basado en componentes



Ventajas

- Reducción de la cantidad de código a generar
- Reducción de riesgos y de costes



Inconvenientes

- Los desarrolladores no pueden garantizar la calidad de todo el producto. Este tiene dependencias externas

El modelo de métodos formales

Comprende un conjunto de actividades que conducen a la especificación matemática del software de computadora.



Ventajas

- Estos modelos garantizan un software prácticamente libre de errores



Inconvenientes

- Suele ser caro y consumir mucho tiempo
- Se requiere de una capacitación detallada
- Es difícil la comunicación con el cliente

Introducción

Debido a la fascinación que provoca la nueva forma de entender los problemas, durante el periodo de **1985 a 1995** surgen unas 50 **metodologías orientadas a objetos** diferentes.



Ejemplos

- El método de **Booch**
- El método de **Rumbaugh** (OMT)
- El método de **Jacobson** (Objectory)
- El método de Coad-Yourdon
- El método de Wirfs-Brock

Introducción

- El proceso unificado nace a partir de la **unificación** de los métodos de **Booch, Jacobson y Rumbaugh**
- Combina las mejores características de cada uno de ellos y genera un nuevo lenguaje de especificación: el **UML** (Unified Modelling Language)
- Este lenguaje **propone** una serie de **diagramas** que servirán como herramienta de **especificación** durante el **proceso de desarrollo** que propone el **proceso unificado**

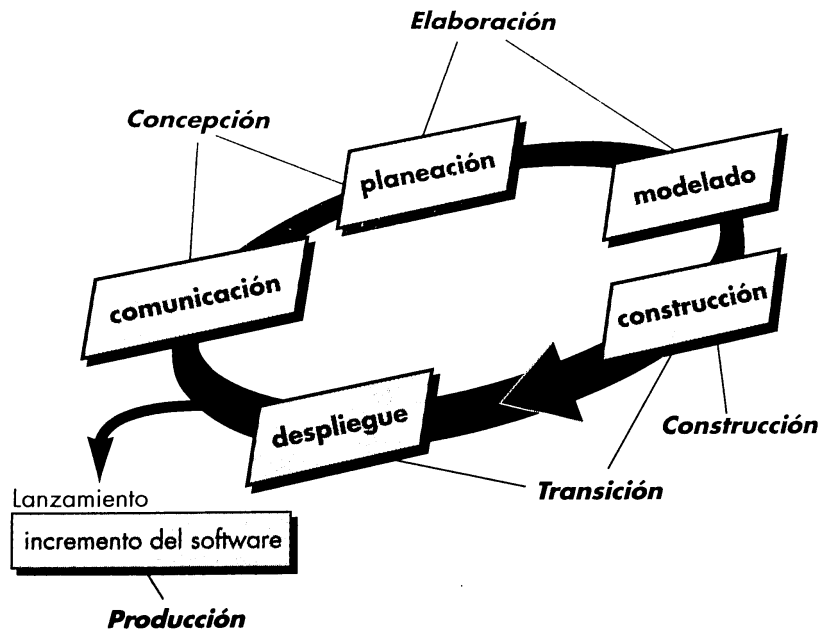
Características

- El **proceso de desarrollo** está basado en un **marco iterativo**
- Este proceso se basa en el **modelo incremental**, con incrementos de una duración **de 2 a 6 semanas**
- Al **final** de este **periodo** se ha de tener un **sistema en funcionamiento**
- **Cada incremento** puede aportar **nueva funcionalidad**, o **mejorar** lo ya **existente**

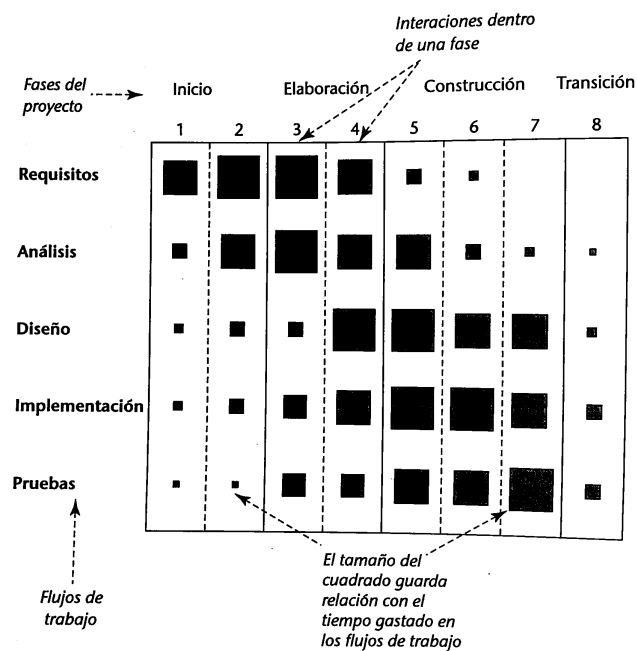
Fases del proceso

- **Concepción**: Visión aproximada, análisis del negocio, alcance, estimaciones imprecisas
- **Elaboración**: Visión refinada, implementación iterativa del núcleo central de la arquitectura, resolución de los riesgos más altos, identificación de más requisitos y alcance, estimaciones más realistas
- **Construcción**: Implementación iterativa del resto de requisitos de menor riesgo y elementos más fáciles, preparación para la puesta en marcha
- **Transición**: Pruebas finales, pruebas de aceptación y otras actividades para la entrega final

Fases del proceso



Fases del proceso



Introducción

- **Problemas** en algunas **metodologías tradicionales**:
 - **Falta de capacidad ante cambios**
 - **Generación de un excesivo volumen de documentación**
- A principios del siglo XXI surgen **estrategias más ligeras** para desarrollo de sistemas donde pueden producirse **cambios importantes** en los **requisitos** durante la vida del proyecto
- La **necesidad** de diseñar sistemas basados en **requisitos acordados** sugiere que la **planificación**, el **análisis** y el **diseño** eficaces, junto con una **documentación** apropiada, tiene un **papel importante** en los proyectos de desarrollo de software

Manifiesto para el Desarrollo de Software Ágil

Es elaborado en **febrero del 2001** por un grupo de diseñadores de software y autores de metodologías:



Nuestra intención es dar a conocer mejores formas para desarrollar software haciendo y ayudando a otros a hacerlo.
En este trabajo vamos a valorar:

- **Individuos e interacciones** sobre procesos y herramientas
- **Software que funcione** sobre una extensa documentación
- **Colaboración del cliente** sobre negociación del contrato
- **Respuesta a los cambios** sobre el seguimiento de un plan

Es decir, aunque los conceptos mostrados a la derecha tienen un alto valor, nosotros valoramos más los situados a la izquierda.

Estrategias ágiles



- Programación Extrema (XP)
- SCRUM
- Crystal Methodologies
- Dynamic Systems Development Method (DSDM)
- Adaptive Software Development (ASD)
- Feature-Driven Development (FDD)
- Lean Development (LD)

¿Qué método es el más adecuado?

- Ningún método funciona de manera universal
- Una de las primeras cuestiones que podemos plantearnos es:
¿utilizar una metodología ágil o una metodología guiada por un plan?
- Depende fundamentalmente de tres factores:
 - El tipo de proyecto
 - La cultura existente en la empresa
 - El conocimiento de herramientas asociadas a las metodologías

Bibliografía



R. Pressman

Software Engineering. A Practitioner's Approach. 8th Ed.

McGraw-Hill, 2014